
Anomaly Detection

Content

- Outliers
- Causes of Outliers
- Anomaly detection techniques
- Outlier detection using Statistics
- Outlier detection using distance based method
- Outlier detection using density based methods
- SMOTE

Concepts

- An **outlier** is a data object that is **markedly different from the other objects in a dataset**.
- **Outliers** are **always defined in the context of other objects** in the dataset.
- A **high-income individual** may be an **outlier in a middle-class** neighborhood dataset, but not in the membership of a luxury vehicle ownership dataset.
- **Outliers** are **rare and stand out** amongst other data points.
- For example, the majority of computer network traffic is legitimate, and a single malicious network attack would be the outlier.

Anomaly Detection

- **Anomaly detection** is the [process of identifying outliers](#) in a dataset that do not conform to the expected behavior.
- It has wide **applications** in **business, scientific, and security domains**.
- **Supervised algorithms** can be used for anomaly detection if the training dataset includes objects with known anomalous outcomes.
- **Unsupervised algorithms** are specialized techniques that can detect outliers without the use of a labeled training dataset.
- Unsupervised algorithms can measure distance from other data points or density around the neighborhood of the data point.

Causes of Outliers

- **Outliers** in a dataset can either be **caused by errors in the data** or **by normal inherent variability**.
- Identifying the source of outliers is important in deciding what action, if any, should be taken.
- **i)Data errors ii) normal variance in the data iii)data from other distribution classes, and iv) distributional assumptions** are common reasons for outliers in a dataset.
- Understanding **why outliers occur** helps in determining the appropriate action to take.
- In some cases, outliers may need to be **isolated and acted upon**, while in others, they **may need to be filtered out** to avoid skewing the final outcome of the analysis.

Causes of Outliers

- **Data errors**
- **Normal variance in the data**
- **Data from other distribution classes**
- **Distributional assumptions** are common reasons for outliers in a dataset.

Causes of Outliers



1. Data errors

- Outliers in a dataset can result from **data errors, such as measurement errors, human errors, or data collection errors.**
- Data points that are obviously errors, like a **height of 1.70 cm** in a dataset of human heights, are **usually ignored** because **they can influence the outcome of a data science task.**
- **Outlier detection** is used as a **preprocessing step** in algorithms such as regression and neural networks to identify and remove these errors.
- **Human errors** that introduce **data errors can be either intentional or unintentional, such as data entry errors or significant bias.**
- **Identifying and removing outliers** caused by data errors is critical to ensure the accuracy of data analysis and modeling.

Causes of Outliers

Normal variance in the data

- In a normal distribution, 99.7% of data points fall within three standard deviations from the mean.
- This means that 0.26% or 1 in 370 data points fall outside of three standard deviations from the mean.
- Outliers are legitimate data points that don't occur frequently but are a part of the dataset.
- **Outliers can skew descriptive statistics like the mean of the dataset**, but they are still valid data points that should not be ignored.

Causes of Outliers

Data from other distribution classes:

- **Daily page views** from user IP addresses typically range from one to several dozen for a customer-facing website.
- **Some IP addresses can have hundreds of thousands of page views** in a day, which could be due to **automated programs or bots**.
- **Bots may be accessing the site legitimately or maliciously**, but it is normal for them to register thousands of page views.
- Bot traffic is considered part of the "**traffic from programs**" class, which is distinct from **traffic from regular browsers** that fall under the **human user class**.

Data from other distribution classes

- Outliers may arise when sensor readings correspond to motion patterns belonging to different activity classes.
- For example, in gyroscope-based HAR datasets, walking typically produces moderate angular velocity values, whereas activities such as running or falling generate significantly higher rotational spikes.
- When such values appear within the walking dataset due to activity transitions or mislabeling, they follow the statistical distribution of another class and therefore appear as outliers.

Causes of Outliers

iv) Distributional assumptions:

- **Outlier data** points can **result from incorrect assumptions** made on the data or distribution.
- For instance, in the case of measuring the **usage of a library in a school**, a **surge in usage during term exams can create an outlier**.
- Similarly, there can be a **surge in retail sales** during a particular day or season, resulting in an outlier in the sales data.
- **Understanding the underlying reasons for outliers is important** to avoid incorrect interpretations of the data and to make appropriate decisions based on the insights gained from the data analysis.

- **Distributional assumption** means that when we analyze data, we often **assume that the data follows a particular statistical distribution**, such as **normal (Gaussian) distribution**.
- If some data points **do not follow this assumed distribution**, they appear as **outliers**, even though they may actually be valid measurements.

Outlier Detection Using Statistical Methods

- Z-score,
- Interquartile Range (IQR)
- Probability Rule

Observation	Gyro Magnitude
1	0.55
2	0.60
3	0.72
4	0.48
5	2.50

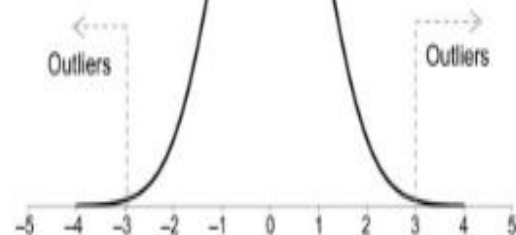


FIGURE 13.1

Standard normal distribution and outliers.

Outlier Detection Using Statistical Methods

Statistical methods assume that data follows a probability distribution.

Many real-world datasets approximately follow a **Gaussian (Normal) distribution**.

Two common statistical techniques are:

- ▶ **Z-score Method**

$$Z = \frac{x - \mu}{\sigma}, \quad \text{if } |Z| > 3$$

the observation is considered a potential outlier.

- ▶ **Interquartile Range (IQR) Method**

$$IQR = Q_3 - Q_1$$

Outlier limits:

$$Q_1 - 1.5/IQR \quad \text{and} \quad Q_3 + 1.5/IQR$$

Values outside this range are classified as outliers.

Statistical Outlier Detection (Z-Score Method)

For a dataset $x_1, x_2, \dots, x_n$:

Mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Standard Deviation

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Z-score

$$Z = \frac{x - \bar{x}}{\sigma}$$

Decision Rule

$$|Z| > 3 \Rightarrow \text{Potential Outlier}$$

Example: Z-Score Outlier Detection

Practical Situation

Time (minutes) taken by students to complete an online quiz:

20, 22, 19, 21, 20, 23, 22, 21, 20, 40

Step 1: Mean

$$\bar{x} = \frac{228}{10} = 22.8$$

Step 2: Standard Deviation

$$\sigma \approx 5.84$$

Step 3: Z-score for $x = 40$

$$Z = \frac{40 - 22.8}{5.84} = 2.94$$

Conclusion

Since $Z \approx 3$, the value 40 minutes is a potential anomaly.

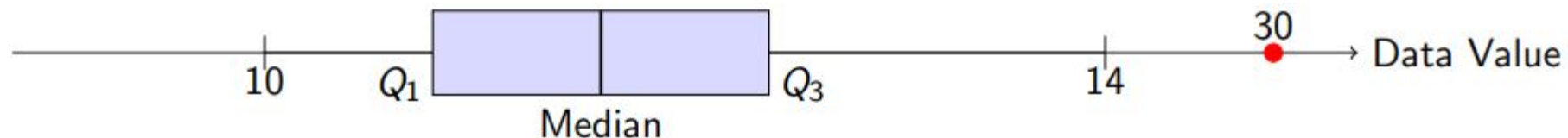
Example: IQR Method

Package weights (kg): 10,11,11,12,12,12,13,13,14,30

Quartiles: $Q_1 = 11$, *Median* = 12, $Q_3 = 13$

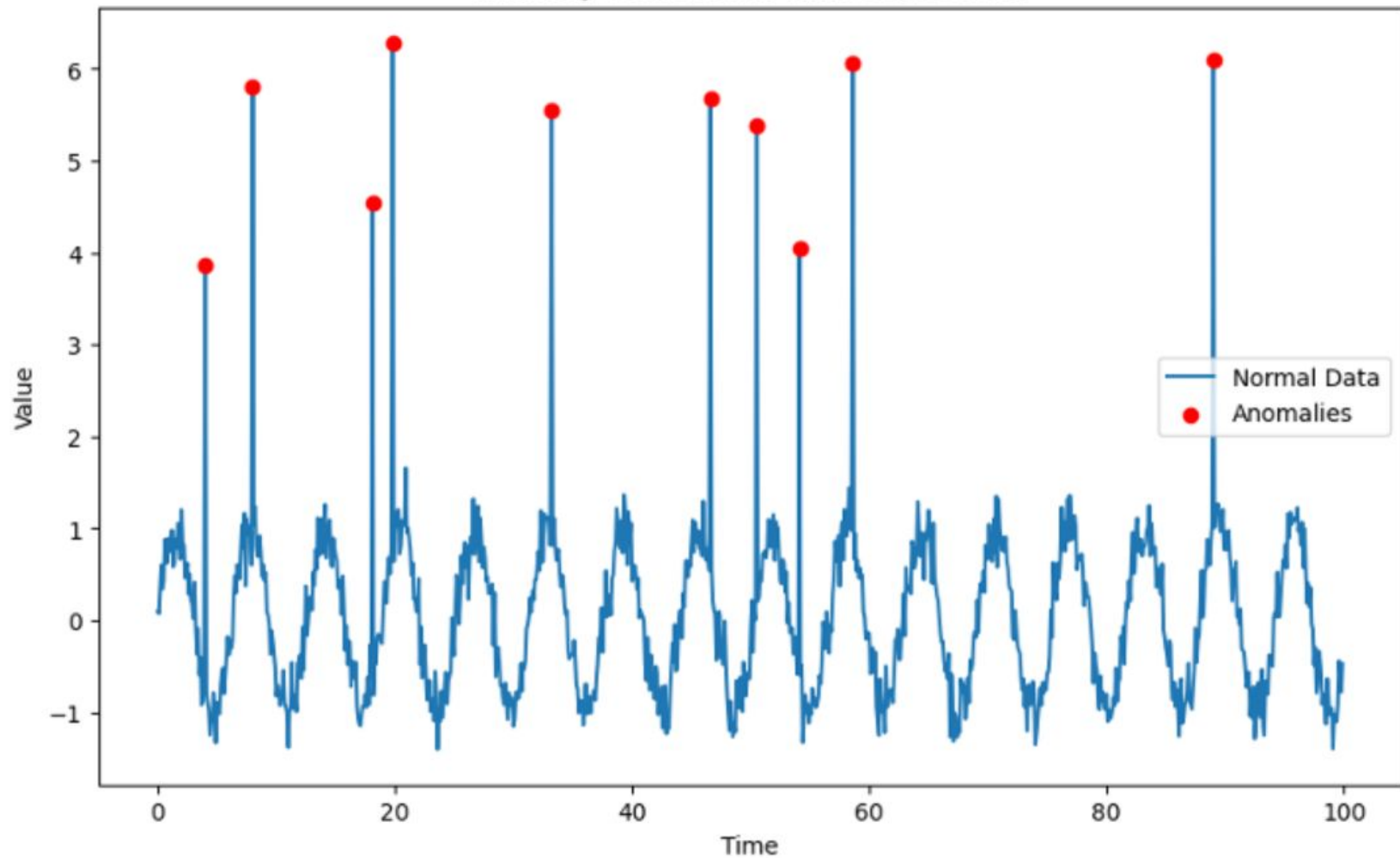
$$IQR = Q_3 - Q_1 = 2$$

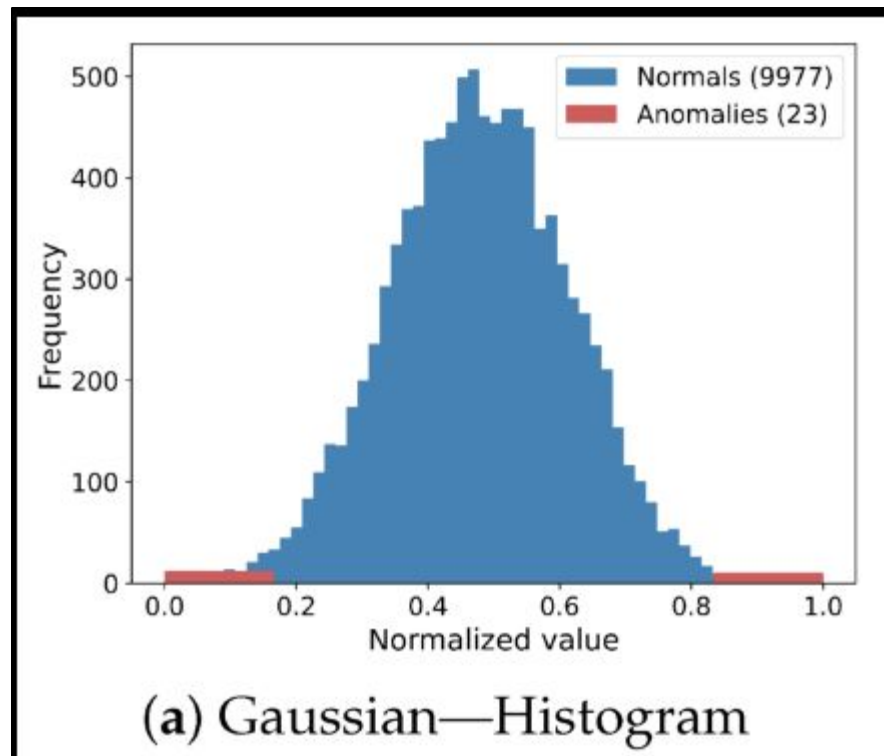
Outlier limits: $Q_1 - 1.5IQR = 8$, $Q_3 + 1.5IQR = 16$



Observation: Value 30 lies beyond the upper limit and is detected as an outlier.

Anomaly Detection in Time-Series Data





Types of Anomalies-Point Anomalies (Global Anomalies)

- A single data point that is very different from the rest of the data.
- If most students score between **50–80 marks**, but one student scores **5**, that score is a **point anomaly**.
- Point anomalies are the simplest and most commonly studied anomalies and are often detected using statistical or distance-based methods.

Contextual Anomalies (Conditional Anomalies)

- A data point that is abnormal **only in a specific context** (time, location, condition).

Temperature = 10°C

- **Abnormal in summer**
- Normal in **winter**
- Contextual anomalies require more advanced models because they involve **two dimensions: behavior + context**.

Collective Anomalies

- A **group of data points** that are normal individually but **abnormal when considered together**.
- One login attempt → normal
- 100 login attempts in 1 minute → anomaly (possible hacking)
- Collective anomalies are important in **time-series data**, cybersecurity, and sensor networks.

Applications of Anomaly Detection

Major uses

Healthcare

- Detection of early stage cancer
- Apps using AI to detect anomaly
- X-ray/CT scans

Defence

- Detecting missile attack
- Chemical attack

Anomaly Detection Techniques

- **Simple Statistical Methods:**

The simplest approach to identifying irregularities in data is to flag the data points that deviate from common statistical properties of a distribution, including mean, median, mode, and quantiles.

- Exploratory data analysis
- Data visualisation

Anomaly Detection Techniques

- **Multivariate anomaly detection**

For identifying anomalies when dealing with one or two variables, data **visualization** can often be a good starting point.

However, when scaling this up to high-dimensional data (which is often the case in practical applications), this approach becomes increasingly difficult. This is fortunately where *multivariate statistics* comes to help.

Multivariate - use ML algorithms

Types of Anomaly detection techniques with example

Data Science based techniques

- Distance based
- Density Based
- Distribution based
- Cluster based
- Classification based

Distance based Outlier Detection

- Distance or proximity-based outlier detection relies on the fact that **outliers are distant from other data points**.
- The proximity measures can be simple **Euclidean distance for real values and cosine or Jaccard similarity** measures for binary and categorical values.

Distance based outlier detection

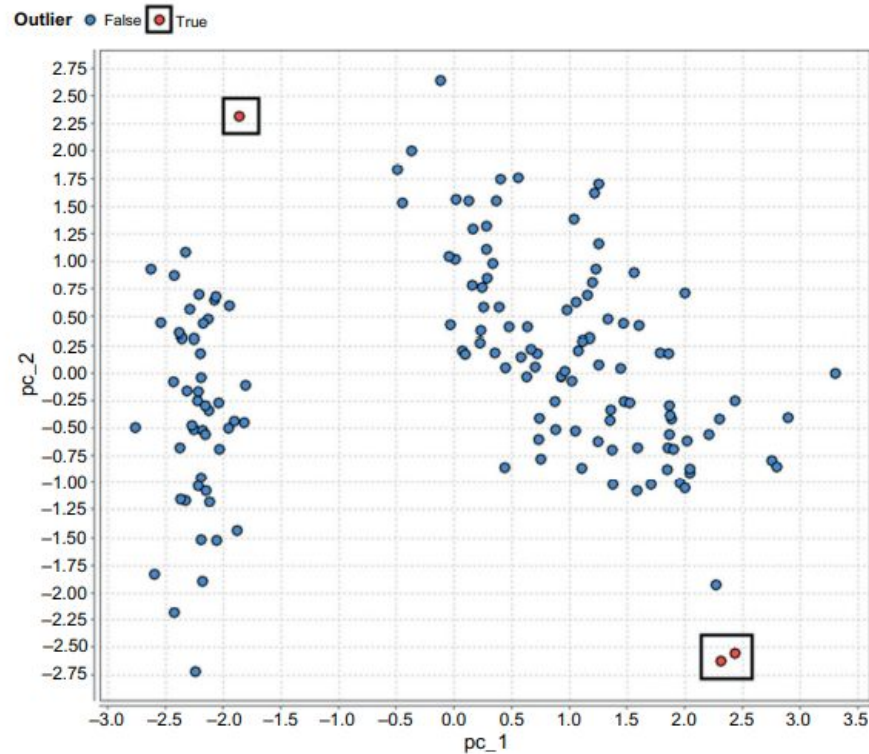


FIGURE 13.3
Dataset with outliers.

Iris dataset

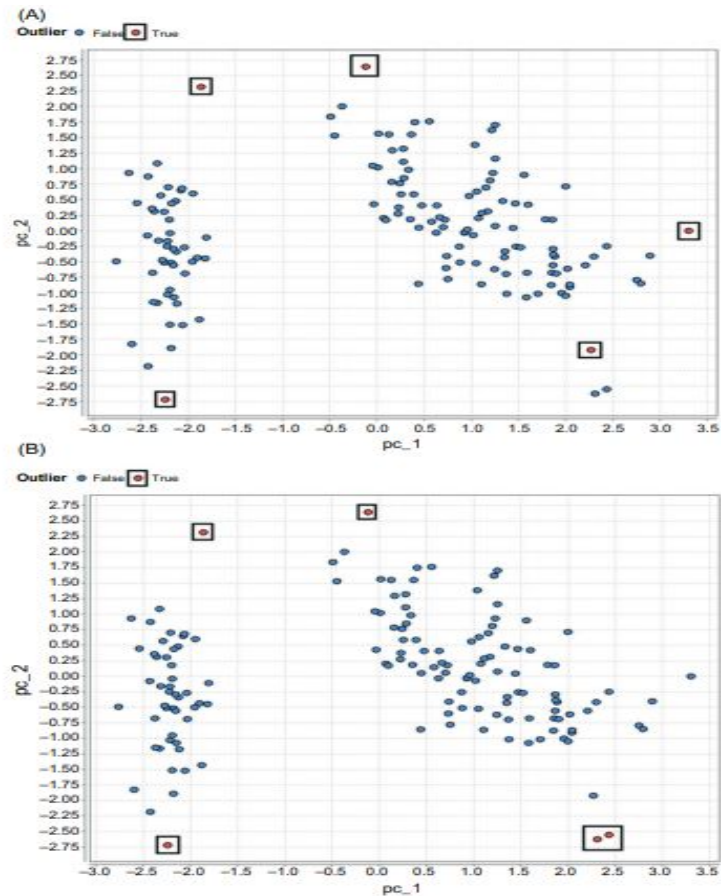


FIGURE 13.4

Top five outliers of Iris dataset when (A) $k = 1$ and (B) $k = 5$.

Distance based Outlier Detection Steps

- Step 1: **Data Preparation** involves normalizing the attributes using the Normalize operator and reducing the number of attributes to two using the PCA operator.
- The PCA operator removes any principal components with a variance threshold more than 0.95.
- These steps are aimed at preparing the data for outlier detection and visualization by normalizing the attributes and reducing the dimensions for easy visualization.

Distance based Outlier Detection Steps

- **Step 2 is to detect outliers using the Detect Outlier (Distances) operator in RapidMiner.**
- This operator has three parameters that can be configured: **the number of neighbors (k), the number of outliers to be found, and the distance function.**
- The number of neighbors parameter specifies **how many neighboring data points are considered** when calculating the density of a point.
- The number of outliers parameter determines **how many outliers are to be found**. The algorithm finds the data points with the highest outlier scores.
- The **distance function parameter specifies the distance measurement function** to be used. In this example, the Euclidean distance function is used.
- The output of the operator is the **example set with an appended outlier attribute**, which has a true or false value indicating whether a data point is an outlier or not.
- For this example, the parameters are set to k=5, number of outliers=10, and Euclidian distance function.

Distance based Outlier Detection Steps

- **Step 3: Execution and Interpretation** involves sorting the result dataset by the outlier attribute (true or false) to identify the specified number of outliers (10 in this case).
- The outliers can be explored in the Chart view of the result set, where the principal components can be set as the X- and Y-axes, and the outlier attribute as the color. The scatterplot shows the outlier data points along with the normal data points.
- Distance-based outlier detection is a simple and widely used algorithm for problems involving many numeric variables. However, the algorithm can become computationally expensive when dealing with high-dimensional datasets with many records, as it needs to calculate distances with other data points in high-dimensional space.

Distance based Outlier Detection Steps

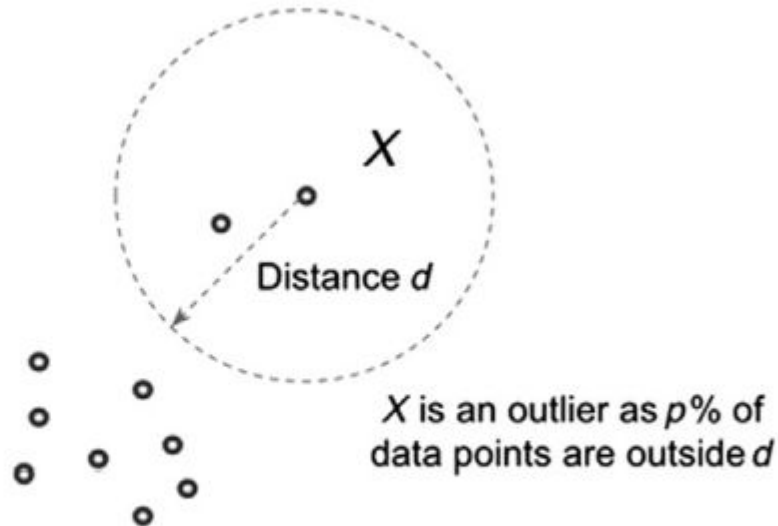


FIGURE 13.7

Outlier detection based on distance and propensity.

Distance-Based Outlier Detection

Distance-based methods identify anomalies by measuring the distance between data points.

Basic Idea

- ▶ Normal data points lie close to each other.
- ▶ Outliers lie far away from the majority of data points.

A distance score can be computed using **k-nearest neighbors (k-NN)**.

$$d_i = \text{distance to the } k^{th} \text{ nearest neighbor}$$

If the distance score of a point is significantly larger than others, it is considered an outlier.

Common distance measures:

- ▶ Euclidean distance
- ▶ Cosine similarity
- ▶ Jaccard distance

Distance-Based Outlier Detection

Distance-based methods classify a point as an outlier if it is far away from other observations.

Distance from Mean

$$d_i = |x_i - \bar{x}|$$

If

$$d_i > \text{threshold}$$

then the observation is considered an outlier.

Example: Distance-Based Method

Trip distances (km):

5, 6, 7, 5, 6, 7, 6, 5, 6, 25

Mean

$$\bar{x} = 7.8$$

Distance of 25

$$d = |25 - 7.8| = 17.2$$

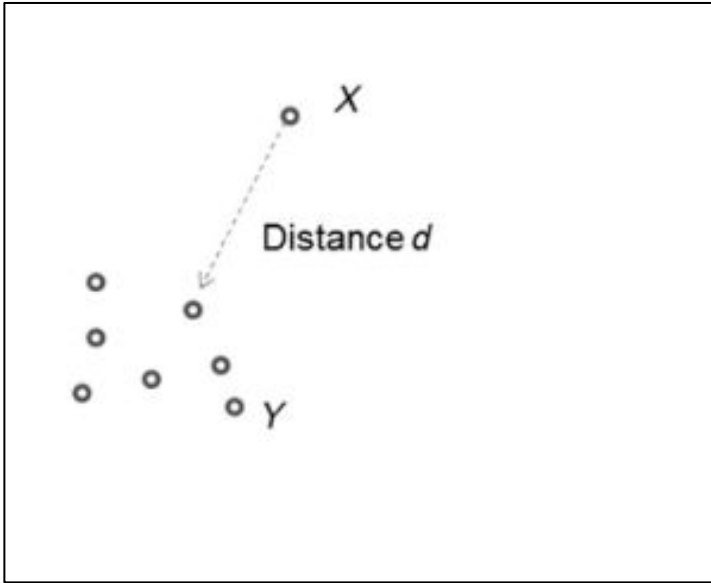
If threshold = 10

$$17.2 > 10$$

Hence 25 km is detected as an outlier.

Outlier Detection Using Data Science

- Distance-based



Distance-Based Outlier Detection using k-NN

Distance-based anomaly detection often uses the **k-nearest neighbors (k-NN)** concept.

For each data point x_i :

- ▶ Compute the distance to its k^{th} nearest neighbor.
- ▶ This distance is called the **distance score**.

$$Score(x_i) = d(x_i, k^{th} \text{ nearest neighbor})$$

Interpretation

- ▶ Small score $\rightarrow$ point lies in dense region
- ▶ Large score $\rightarrow$ point lies far from other data
- ▶ Large score $\Rightarrow$ potential outlier

Effect of Parameter k in Distance-Based Detection

The choice of k strongly influences the results.

If $k = 1$

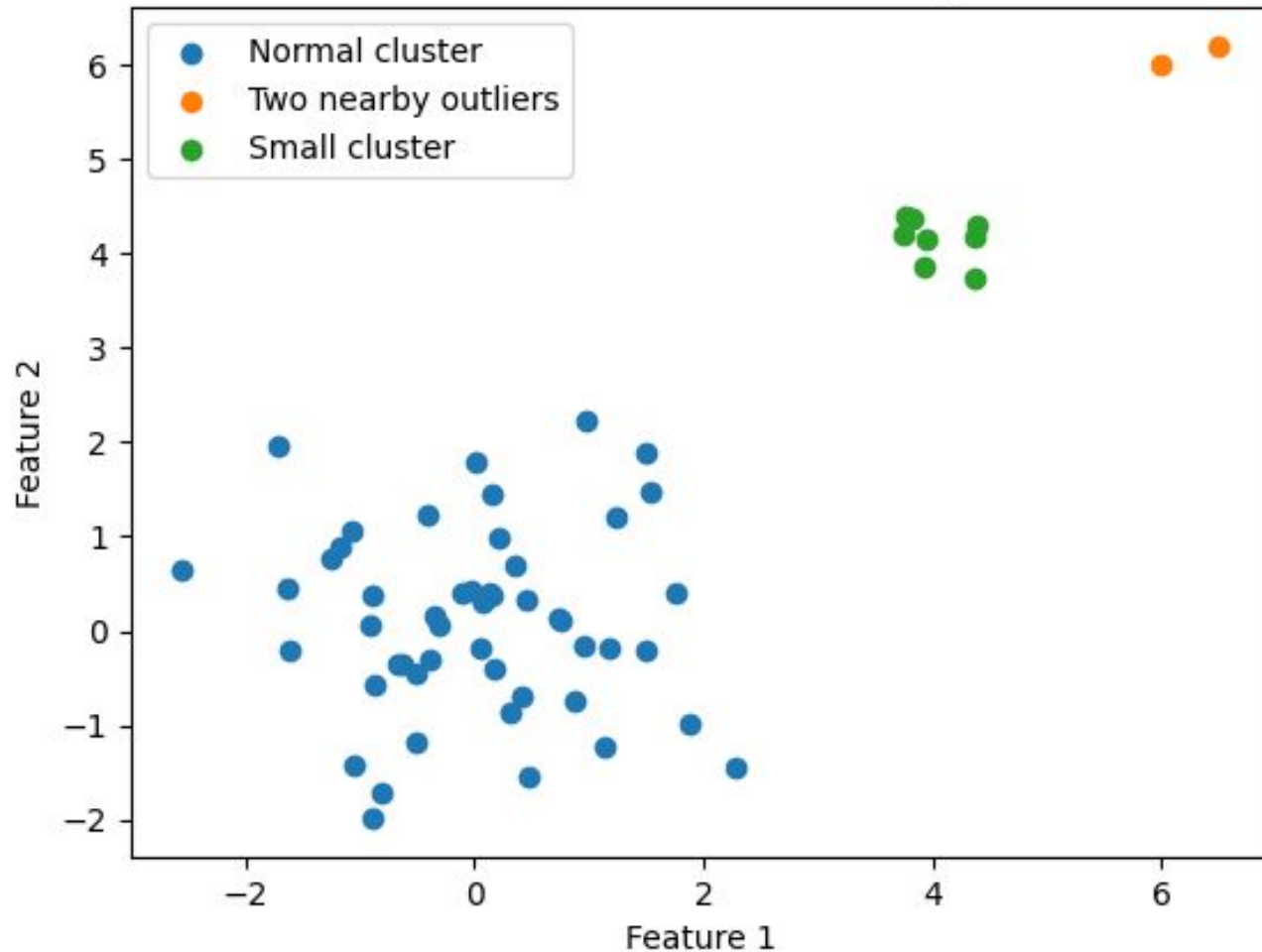
- ▶ Two nearby outliers may appear normal
- ▶ Each outlier becomes the nearest neighbor of the other

If k is very large

- ▶ Small clusters may incorrectly appear as outliers
- ▶ The algorithm becomes insensitive to local structure

Therefore choosing an appropriate value of k is important.

Effect of k in Distance-Based Anomaly Detection



Choosing k is a **balance problem**:

- Too small $\rightarrow$ misses anomalies
- Too large $\rightarrow$ creates false anomalies

$k = 1$ $\rightarrow$ judging a student by only their best friend

large k $\rightarrow$ judging a student by the entire school

Example: Distance-Based Outlier Detection using k-NN

Consider the following one-dimensional dataset:

2, 3, 3, 4, 4, 5, 5, 6, 6, 20

Let

$$k = 2$$

Distance-based anomaly detection computes the distance from each point to its k^{th} **nearest neighbor**.

$$Score(x_i) = d(x_i, x_{(k)})$$

where $x_{(k)}$ is the k^{th} nearest neighbor.

Since the data is one-dimensional, distance is computed as

$$d(x_i, x_j) = |x_i - x_j|$$

Step 1: Compute k-NN Distance for Normal Points

Example for point 2

Neighbor	Distance
3	1
3	1
4	2
4	2

Sorted distances:

1, 1, 2, 2

$$\text{Score}(2) = 1$$

Compute k-NN Distance for Normal Points

Similarly,

Point	2nd Nearest Distance
3	1
4	1
5	1
6	1

These points lie in a dense cluster, so their k-NN distance is small.

Step 2: Compute k-NN Distance for Point 20

Distances from point 20

Neighbor	Distance
6	14
6	14
5	15
5	15

Sorted distances:

14, 14, 15, 15

The 2nd nearest neighbor distance is

$$\text{Score}(20) = 14$$

Step 3: Identify the Outlier

Point	kNN Score
2	1
3	1
4	1
5	1
6	1
20	14

Observation:

- ▶ Most points have very small distance scores.
- ▶ Point 20 has a much larger score.

$$14 \gg 1$$

Therefore,

20 is detected as an outlier

Data Science based techniques

- **Density-based methods:** These methods identify **outliers** based on their **low density** in a neighborhood relative to regular data points. Outliers are often located in low-density areas, while regular data points cluster in high-density areas. The density of a data point is inversely related to its distance to its neighbors.
- **Distribution-based methods:** These methods use **statistical distributions** to identify data points with **low probability of occurrence**, which are likely to be outliers. **Data points that fall outside a certain range or threshold** can be considered outliers. A simple normal distribution can be used to model the dataset, and outliers can be identified by calculating the mean and standard deviation.

Data Science based techniques

- **Clustering methods:** These methods identify **outliers as data points that do not form tight clusters** or are far away from other clusters. Most clustering algorithms have a minimum threshold of data points required to form a cluster, so outliers are often **alone data points that are not clustered**. Even if outliers form a cluster, they are usually far away from other clusters.
- **Classification techniques:** Nearly **all classification techniques can be used to identify outliers** if previously labeled data is available. In these techniques, a known test dataset is used where one of the class labels is "Outlier." The outlier-detection classification model is built based on this test dataset and can pre

DENSITY-BASED OUTLIER DETECTION

- **Outliers** are data points that **occur less frequently** than normal data points and tend to occupy low-density areas in the data space.
- **Density** is a **measure of the number of data points in a normalized unit of space** and is **inversely proportional to the distances between data points**.
- **Density-based outlier algorithms** aim to **identify data points in low-density areas**. They can **assign outlier scores based on the inverse of the average distance of all k neighbors** or by calculating the **number of data points within a normalized unit distance**.
- The approach for density-based outliers is similar to that of **density-based clustering and k-NN classification algorithms**.
- Density-based outlier detection can handle datasets with high dimensions and is efficient in detecting local outliers, but it may not be effective in identifying global outliers that are far away from dense areas.

Anomaly Detection

- **Clustering techniques** can be leveraged for anomaly detection, where the **outlier forms a separate cluster** from other clusters because they are far away from other data points.
- It is important to understand the **reason why such data points occur** in a dataset to effectively **identify and act** on the **results of outlier detection**.

2. DENSITY-BASED OUTLIER DETECTION

- How it works
- Since **distance is the inverse of density**, the approach of a density-based outlier can be explained with two parameters:
 - distance (d) and
 - proportion of data points (p).
 - A **point X** is considered an **outlier** if **at least p fraction of points lie more than d distance from the point**

Comparison

Aspect	Distance-based	Density-based
Data points classification	Outlier if farther from neighbors	Outlier if in low-density area
Density calculation	Not required	Required
Effect of data dimensionality	Slower as dimensionality increases	Slower as dimensionality increases
Robustness to noise	Less robust	More robust
Handling of different density levels	Limited	Capable of handling varying density levels
Determination of parameter value(s)	Relatively easier	Requires tuning of multiple parameters
Suitable for	Low to medium-dimensional data with small clusters of outliers	High-dimensional data with large clusters of outliers and varying density levels

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN groups points into clusters based on density and identifies isolated points as anomalies.

Key Parameters

- ▶ ϵ : neighborhood radius
- ▶ *minPts* : minimum number of neighbors required to form a dense region

Types of Points

- ▶ **Core Point** : has at least *minPts* neighbors within radius ϵ
- ▶ **Border Point** : lies within the neighborhood of a core point but has fewer neighbors
- ▶ **Noise Point (Outlier)** : has very few or no neighbors within ϵ

Advantage

- ▶ Can detect clusters of arbitrary shapes
- ▶ Automatically identifies noise points (outliers)

Density-Based Outlier Detection (DBSCAN)

Density-based methods classify points based on the number of neighbors.

- ▶ Points in dense regions form clusters.
- ▶ Points with very few neighbors are classified as outliers.

Cluster 1



Cluster 2



Outlier



DBSCAN Principle

- ▶ Define neighborhood radius ϵ
- ▶ Minimum neighbors *minPts*
- ▶ Points with neighbors $< minPts$ are labeled as anomalies

Example: Density-Based Detection

Transaction amounts (Rs.):

100, 105, 98, 110, 95, 102, 107, 103, 99, 500

Using neighborhood radius:

$$\epsilon = 15$$

Most values lie within a dense cluster around 100.

Distance of 500 from others is very large.

Therefore 500 has no nearby neighbors and is classified as an anomaly.

Example: Local Outlier Factor (LOF)

Consider the following 2D data points representing transaction patterns:

$$A(1, 1), B(1.2, 1.1), C(0.9, 1.0), D(1.1, 0.9), E(5, 5)$$

Clearly, points A, B, C, D form a dense cluster while point E is far away.

Let the number of nearest neighbors be

$$k = 2$$

Step 1: k-distance and Neighbors

Distances from point $E(5, 5)$:

$$d(E, A) = \sqrt{(5 - 1)^2 + (5 - 1)^2} = \sqrt{32} = 5.66$$

$$d(E, B) = \sqrt{(5 - 1.2)^2 + (5 - 1.1)^2} \approx 5.45$$

$$d(E, C) \approx 5.73$$

$$d(E, D) \approx 5.73$$

The two nearest neighbors of E are:

B, A

Step 2: Local Density Concept

LOF compares the density around a point with the density of its neighbors.

- ▶ Points A, B, C, D are close to each other
- ▶ Their neighborhood distances are small
- ▶ Therefore their **local density is high**

For point E :

- ▶ Distance to neighbors is very large
- ▶ Local density around E is very low

Step 3: LOF Interpretation

The Local Outlier Factor is defined as

$$LOF(p) = \frac{\text{average density of neighbors}}{\text{density of point } p}$$

Interpretation:

- ▶ $LOF \approx 1$: normal point
- ▶ $LOF > 1$: possible anomaly
- ▶ $LOF \gg 1$: strong outlier

For point E :

$$LOF(E) \gg 1$$

Therefore point $E(5, 5)$ is classified as an **outlier**.

Synthetic Minority Oversampling Technique (SMOTE)

Content

- Synthetic Minority Oversampling Technique
- Imbalanced-Learn Library
- SMOTE for Balancing Data
- SMOTE for Classification
- SMOTE With Selective Synthetic Sample Generation
 - Borderline-SMOTE
 - Borderline-SMOTE SVM
 - Adaptive Synthetic Sampling (ADASYN)

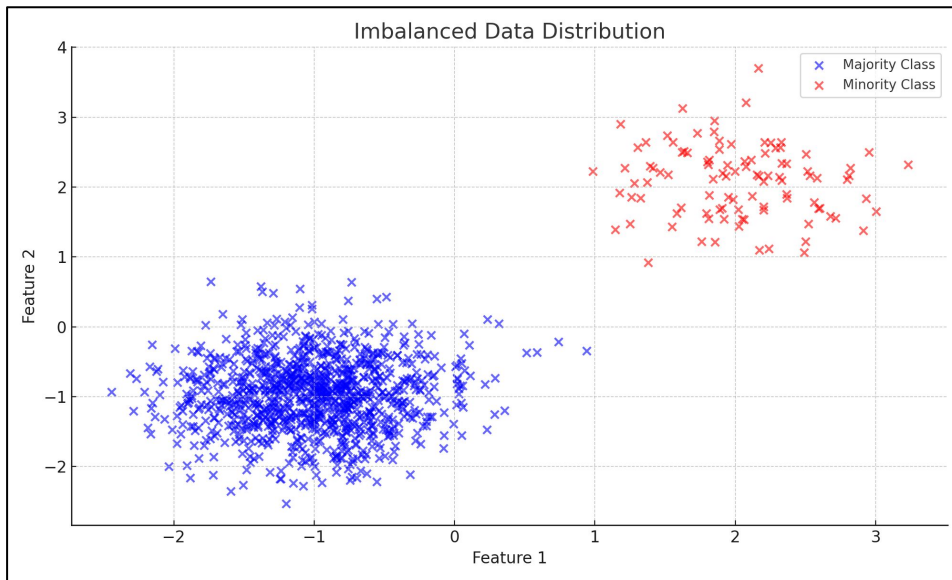
SMOTE

- **Imbalanced classification** involves severe class imbalance, which can lead to **poor performance on the minority class**.
- **Oversampling the minority class** is one way to address this issue.
- **SMOTE** is a popular oversampling method that **generates new examples for the minority class** by interpolating between existing examples.
- **SMOTE** can **improve model performance on the minority class** without introducing bias.
- **SMOTE** is a useful tool for **improving the accuracy of predictive models** on **imbalanced datasets**.

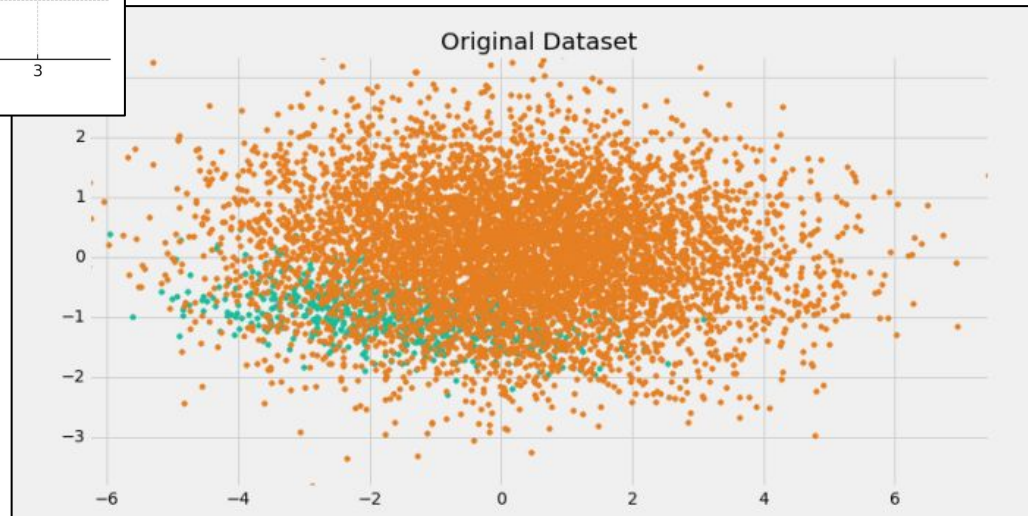
Content

- How the SMOTE synthesizes new examples for the minority class.
- How to correctly fit and evaluate machine learning models on SMOTE-transformed training datasets.
- How to use extensions of the SMOTE that generate synthetic examples along the class decision boundary.

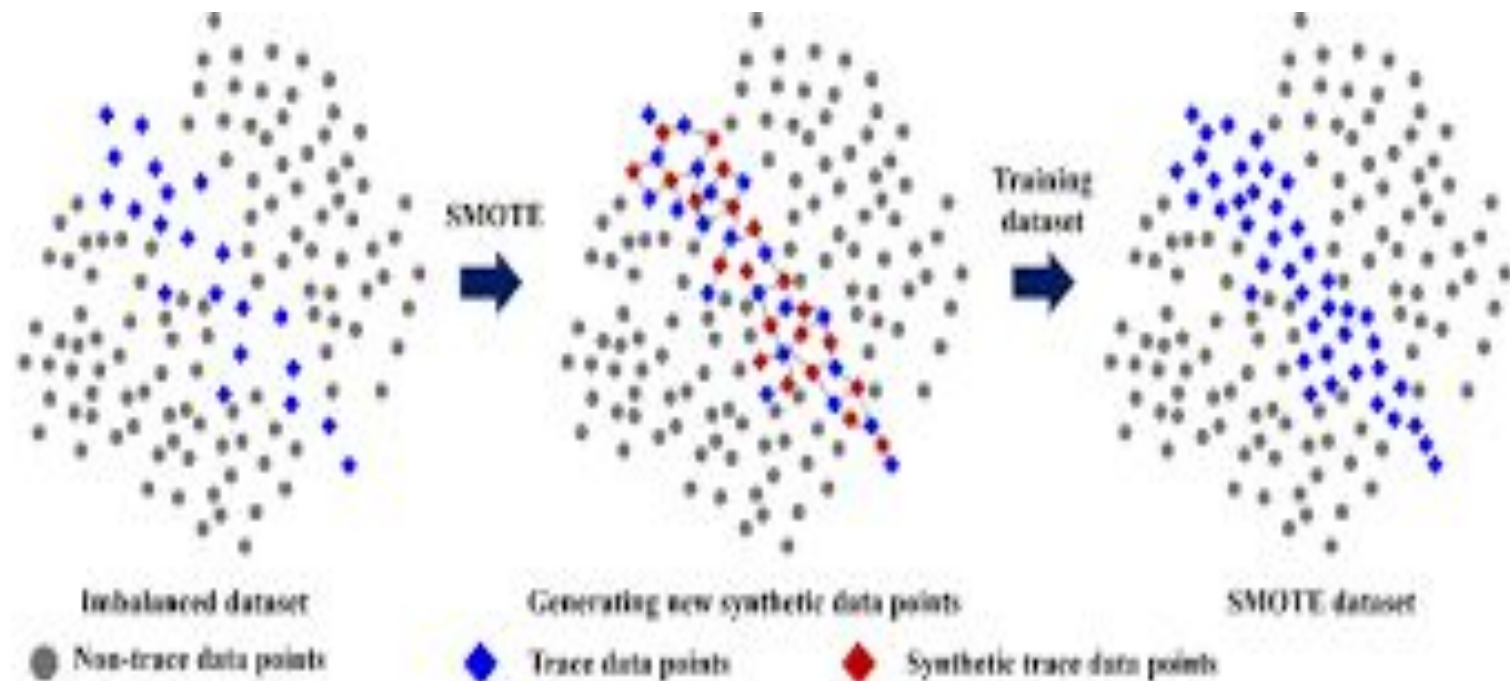
[link](#)

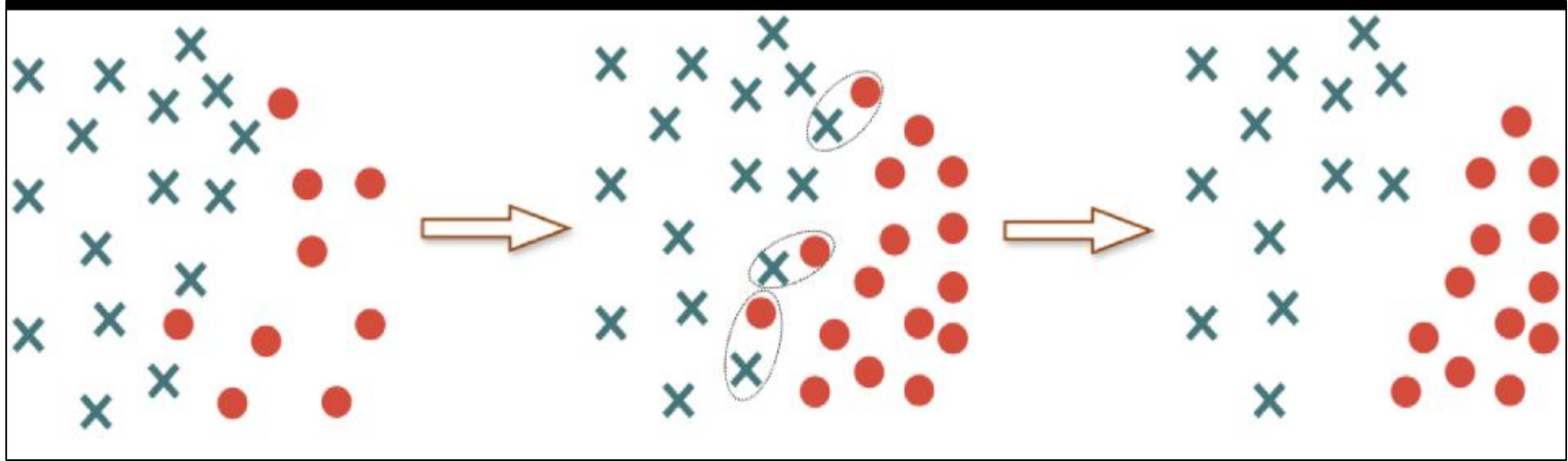


- Model becomes biased
- Minority class is misclassified



Basic Smote

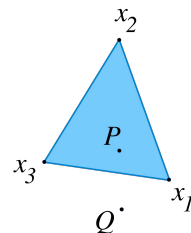
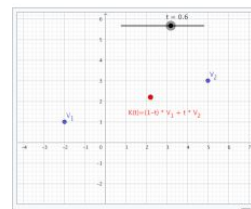




- They are **interpolated points**
- Dataset becomes more balanced

Synthetic Minority Oversampling Technique

- **SMOTE** first selects a minority class instance **a** at random and finds its **k** nearest minority class neighbors.
- The synthetic instance is then created by choosing **one of the k nearest neighbors b** at random and connecting a and b to form a line segment in the feature space. The synthetic instances are generated as a **convex combination** of the two chosen instances **a** and **b**.
- Use SMOTE to **oversample the minority class** to balance the class distribution.
- A general downside of the approach is that **synthetic examples are created without considering the majority class**, possibly resulting in **ambiguous examples** if there is a strong overlap for the classes.



SMOTE generates data everywhere, but should we really generate data everywhere?

Or only where it is needed? That's where

Borderline-SMOTE and ADASYN come into picture.

Imbalanced-Learn Library

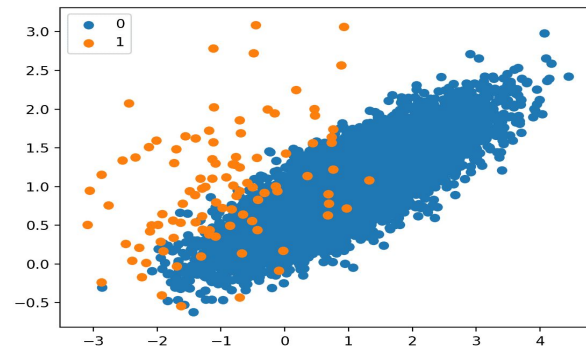
An open-source Python library that provides a variety of techniques for addressing class imbalance in machine learning datasets, including SMOTE.

```
sudo pip install imbalanced-learn
```

```
# check version number
import imblearn
print(imblearn.__version__)
```

SMOTE for Balancing Data

- SMOTE can be **used to balance the class distribution in imbalanced datasets**, which can improve the performance of classifiers trained on such datasets.
- Dataset with 10,000 examples and a 1:100 class distribution.
- Confirms the 1:100 ratio, in this case with about 9,900 examples in the majority class and 100 in the minority class.



Scatter Plot of Imbalanced Binary Classification Problem

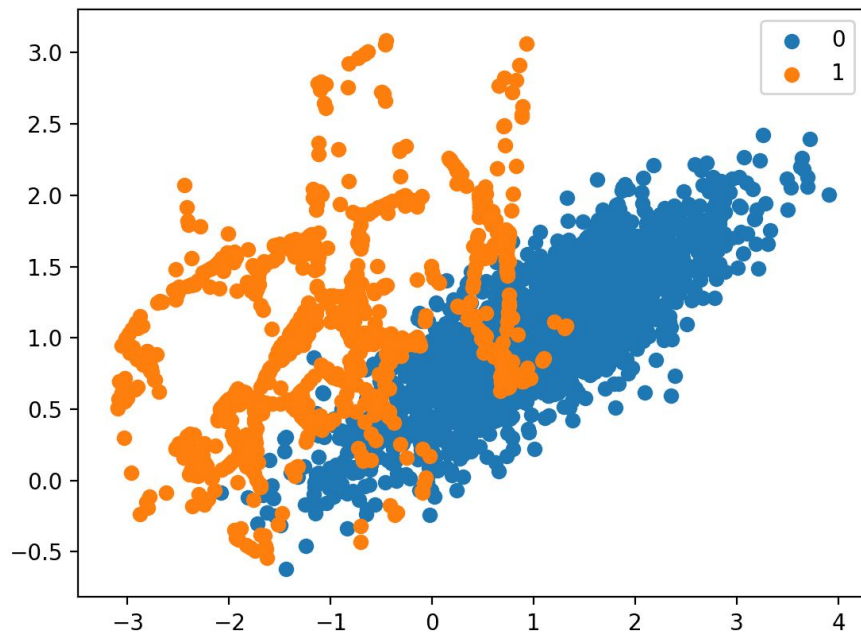
SMOTE for Classification:

- SMOTE is primarily used in classification problems where the minority class is of interest, and the **goal is to improve the classifier's ability to correctly classify instances of the minority class.**

SMOTE

```
from collections import Counter
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE
from matplotlib import pyplot
from numpy import where
# define dataset
X, y = make_classification(n_samples=10000, n_features=2, n_redundant=0,
n_clusters_per_class=1, weights=[0.99], flip_y=0, random_state=1)
# summarize class distribution
counter = Counter(y)
print(counter)
# transform the dataset
oversample = SMOTE()
X, y = oversample.fit_resample(X, y)
# summarize the new class distribution
counter = Counter(y)
print(counter)
# scatter plot of examples by class label
for label, _ in counter.items():
    row_ix = where(y == label)[0]
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1], label=str(label))
pyplot.legend()
pyplot.show()
```


Basic SMOTE



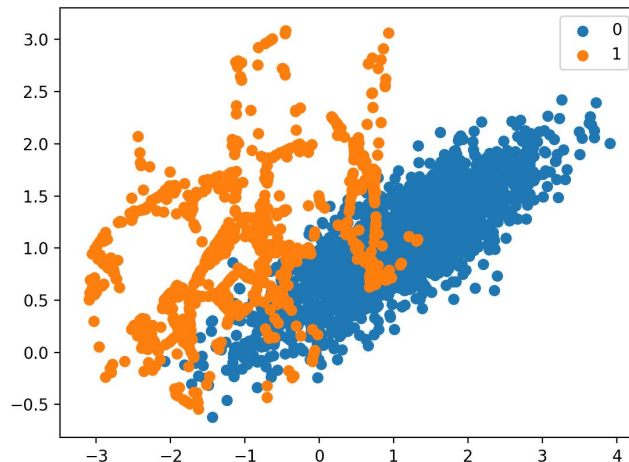
Imbalanced Dataset Transformed by SMOTE and Random Undersampling

- The dataset is transformed, first by oversampling the minority class, then undersampling the majority class.
- The final class distribution after this sequence of transforms matches our expectations with a 1:2 ratio or about 2,000 examples in the majority class and about 1,000 examples in the minority class.

```
over = SMOTE(sampling_strategy=0.1)
under = RandomUnderSampler(sampling_strategy=0.5)
```

We can then chain these two transforms together into a [Pipeline](#).

```
steps = [('o', over), ('u', under)]
pipeline = Pipeline(steps=steps)
# transform the dataset
X, y = pipeline.fit_resample(X, y)
```

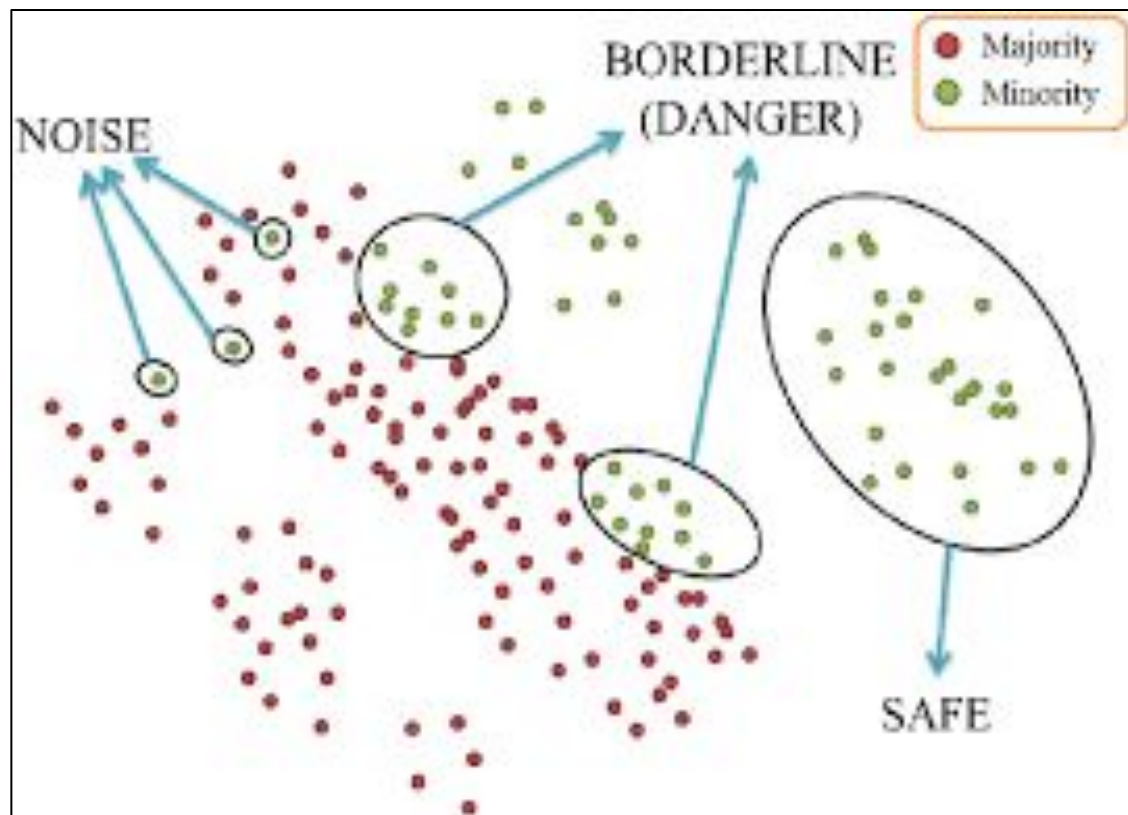


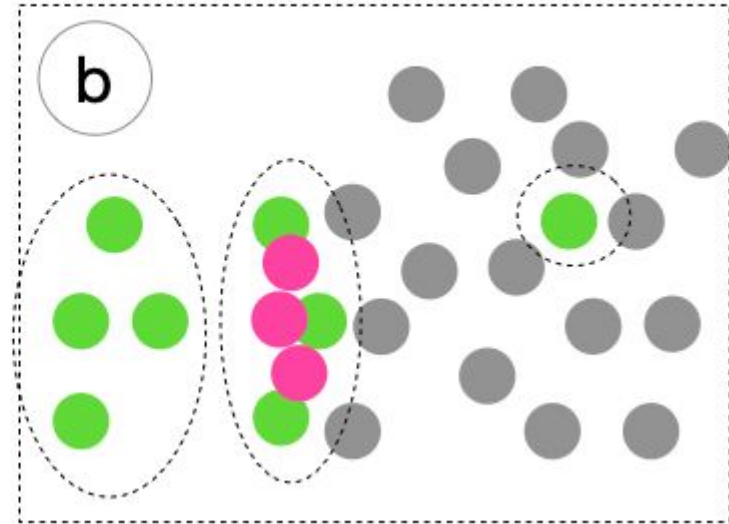
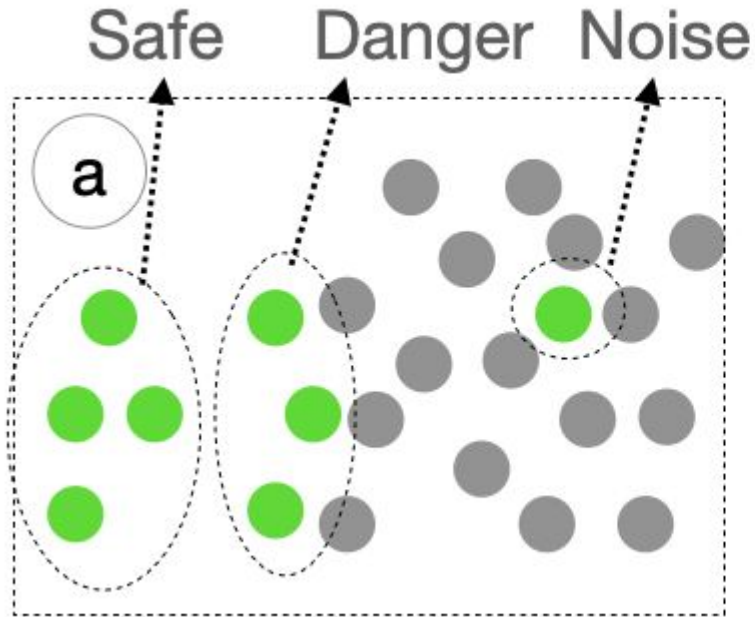
SMOTE With Selective Synthetic Sample Generation

- A modification of the SMOTE algorithm that attempts to **address the issue of creating ambiguous synthetic examples** by selectively generating synthetic samples only in areas of the feature space that are sparsely populated by the minority class.

Borderline-SMOTE

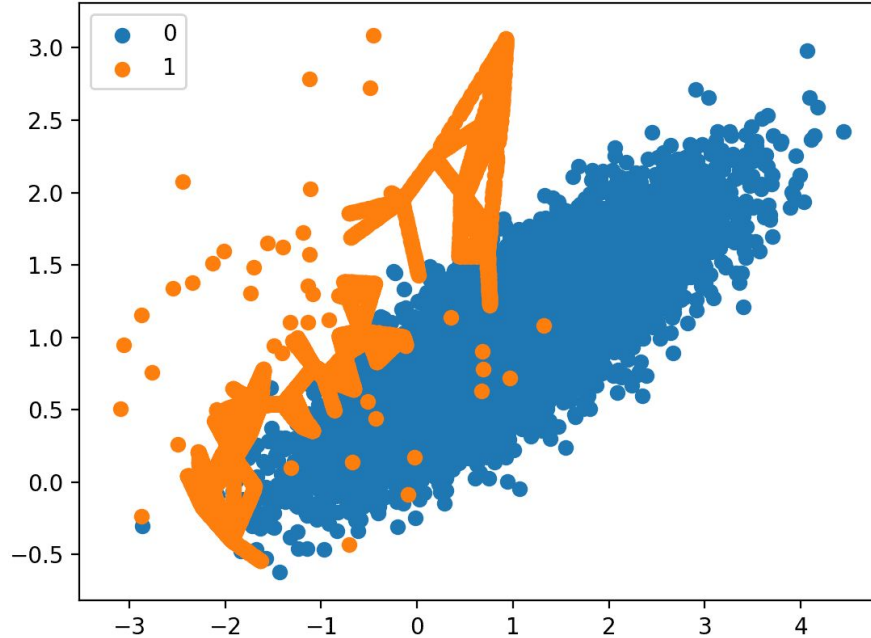
- A popular extension to SMOTE involves selecting those instances of the minority class that are misclassified, such as with a k-nearest neighbor classification model.
- Borderline-SMOTE: A variant of SMOTE that **focuses on the borderline instances of the minority class**, which are those that are closest to the majority class. This technique can be more effective than standard SMOTE in some cases.
- **A version** of the method that **also oversampled the majority class** for those examples that cause a misclassification of borderline instances in the minority class. This is referred to as Borderline-SMOTE1,
- Whereas the **oversampling of just the borderline cases in minority class** is referred to as Borderline-SMOTE2.





Borderline-SMOTE is smarter than SMOTE because it strengthens the weak decision boundary instead of filling the entire space.

Borderline-SMOTE



Scatter Plot of Imbalanced Dataset With Borderline-SMOTE Oversampling

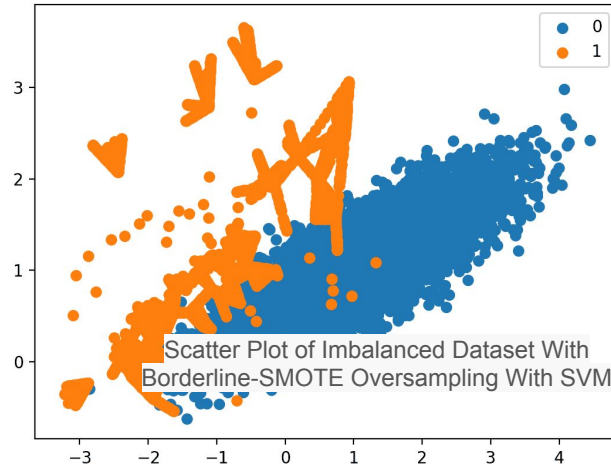
Now the question is – how do we accurately find the boundary?

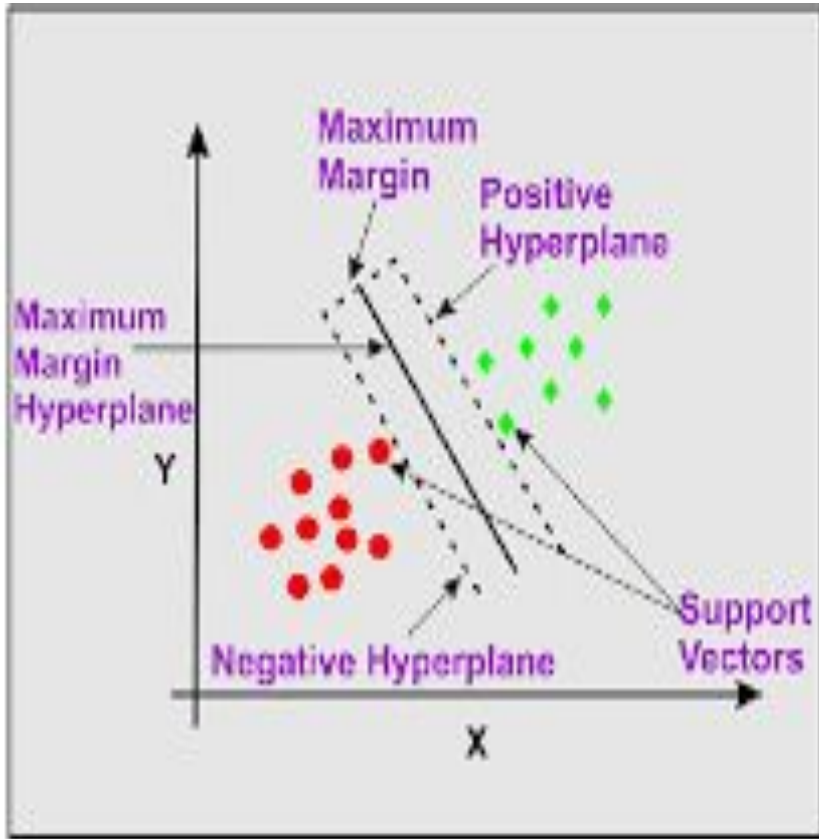
Instead of estimating, can we mathematically learn it?

That leads us to Borderline-SMOTE with SVM

Borderline-SMOTE SVM:

- An extension of the Borderline-SMOTE technique that **incorporates support vector machines (SVMs) to guide the selection of synthetic examples.**
- **Borderline-SMOTE SVM uses an SVM classifier to identify the exact decision boundary and generates synthetic samples near that boundary**





Step 1:

Train an **SVM classifier**

Step 2:

Find:

Support vectors (boundary points)

Step 3:

Identify minority support vectors

Step 4:

Generate synthetic samples:

Near these support vectors

Along boundary region

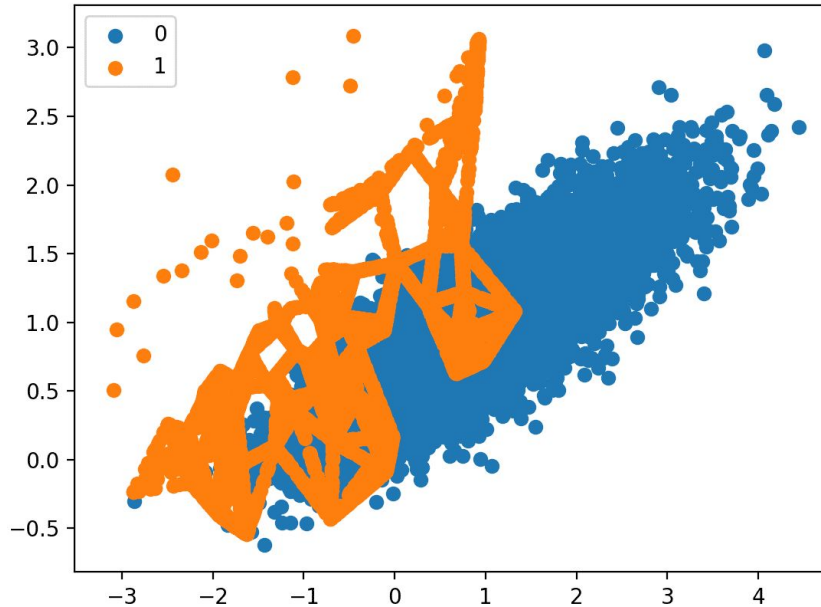
So far, we focused on boundary... but what if some regions are harder to learn than others?

Can we generate more data in difficult regions? That's where ADASYN comes in.

Adaptive Synthetic Sampling (ADASYN)

- Generating synthetic samples inversely proportional to the density of the examples in the minority class.
- ADASYN (Adaptive Synthetic Sampling) generates more synthetic data in regions where the model finds it difficult to learn.
- **Generate more synthetic examples** in regions of the feature space where the **density of minority examples is low**, and **fewer or none** where the **density is high**.
- *More synthetic data is generated for **minority class samples that are harder to learn** compared to those minority samples that are easier to learn.*

Adaptive Synthetic Sampling (ADASYN)



Scatter Plot of Imbalanced Dataset With Adaptive Synthetic Sampling (ADASYN)

Anomaly

- Anomaly is a pattern in the data that does not conform to the expected behavior
- Also referred to as outliers, exceptions, peculiarities, surprise, etc.
- Anomalies translate to significant (often critical) real life entities
 - Cyber intrusions
 - Credit card fraud

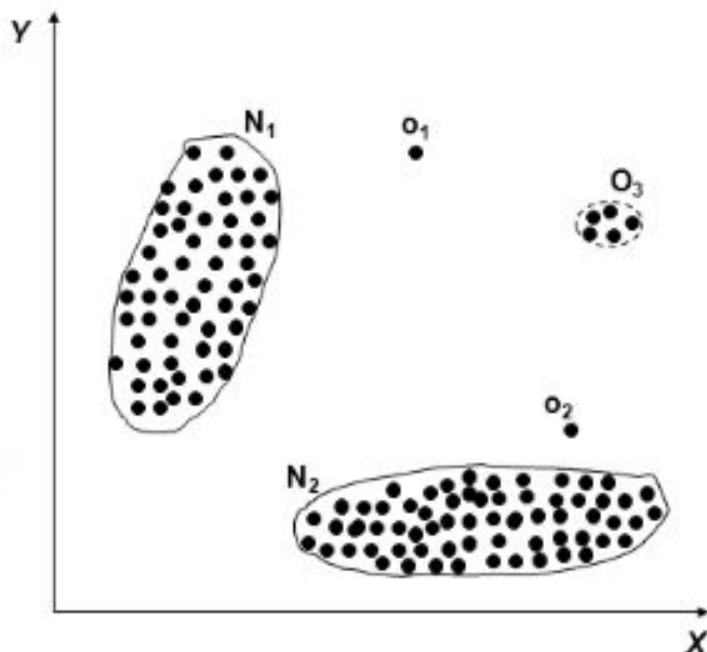
Real World Anomalies

- Credit Card Fraud
 - An abnormally high purchase made on a credit card
- Cyber Intrusions
 - A web server involved in *ftp* traffic



Simple Example

- N_1 and N_2 are regions of normal behavior
- Points o_1 and o_2 are anomalies
- Points in region O_3 are anomalies



Applications of Anomaly Detection

- Network intrusion detection
- Insurance / Credit card fraud detection
- Healthcare Informatics / Medical diagnostics
- Industrial Damage Detection
- Image Processing / Video surveillance
- Novel Topic Detection in Text Mining

Intrusion Detection

- **Intrusion Detection:**
 - Process of monitoring the events occurring in a computer system or network and analyzing them for intrusions
 - Intrusions are defined as attempts to bypass the security mechanisms of a computer or network
- **Challenges**
 - Traditional signature-based intrusion detection systems are based on signatures of known attacks and cannot detect emerging cyber threats
 - Substantial latency in deployment of newly created signatures across the computer system
- **Anomaly detection can alleviate these limitations**



Fraud Detection

- **Fraud detection refers to detection of criminal activities occurring in commercial organizations**
 - Malicious users might be the actual customers of the organization or might be posing as a customer (also known as identity theft).
- **Types of fraud**
 - Credit card fraud
 - Insurance claim fraud
 - Mobile / cell phone fraud
 - Insider trading
- **Challenges**
 - Fast and accurate real-time detection
 - Misclassification cost is very high



Healthcare Informatics

- Detect anomalous patient records
 - Indicate disease outbreaks, instrumentation errors, etc.
- Key Challenges
 - Only normal labels available
 - Misclassification cost is very high
 - Data can be complex: spatio-temporal



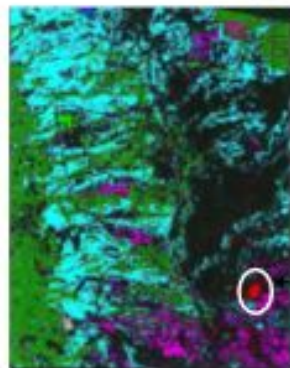
Industrial Damage Detection

- Industrial damage detection refers to detection of different faults and failures in complex industrial systems, structural damages, intrusions in electronic security systems, suspicious events in video surveillance, abnormal energy consumption, etc.
 - Example: Aircraft Safety
 - Anomalous Aircraft (Engine) / Fleet Usage
 - Anomalies in engine combustion data
 - Total aircraft health and usage management
- Key Challenges
 - Data is extremely huge, noisy and unlabelled
 - Most of applications exhibit temporal behavior
 - Detecting anomalous events typically require immediate intervention



Image Processing

- Detecting outliers in a image monitored over time
- Detecting anomalous regions within an image
- Used in
 - medical image analysis
 - video surveillance
 - satellite image analysis
- Key Challenges
 - Detecting collective anomalies
 - Data sets are very large



Anomaly

Online Anomaly Detection

- Data in many rare events applications arrives continuously at an enormous pace
- There is a significant challenge to analyze such data
- Examples of such rare events applications:

- Video analysis



- Network traffic monitoring

- Aircraft safety



- Credit card fraudulent transactions

